

Realized Volatility Forecasting

A Project Reference

Everything needed to build and defend
the vol forecasting system

Ryan Vincent

Contents

I	The Project	4
1	What We're Forecasting	5
1.1	The Universe	5
1.2	Success Criteria	5
1.3	The High-Level Pipeline	6
2	Our Data	7
2.1	Data Sources	7
2.2	GS Edge vs. Public Data	7
2.3	Constraints That Shape Decisions	7
II	The Feature Set	9
3	HAR Core and Measurement Quality	10
3.1	The Five Foundation Features	10
3.2	The HARQ Shrinkage Mechanism	10
3.3	Baseline Performance	11
4	Asymmetric Volatility	12
4.1	Features	12
4.2	The Leverage Effect	12
4.3	Continuous vs. Jump Variation	12
4.4	Cumulative Performance	13
5	Options-Implied Features	14
5.1	Features	14
5.2	Horizon Dependence	14
5.3	What We Compute	15
5.4	Cumulative Performance	15
6	Microstructure Features	16
6.1	Features	16
6.2	The Optiver Evidence	16
6.3	Engineering Principle	16
6.4	Data Constraints	17
7	Cross-Asset Spillovers	18
7.1	Features	18
7.2	Impact	18
7.3	Graph-HAR	18
7.4	GS Advantage	19

8	Feature Composition and Selection	20
8.1	Calendar, Memory, and Sentiment (Layers 5–7)	20
8.2	The Diminishing Returns Curve	21
8.3	Feature Priority by Forecast Horizon	21
8.4	Feature Engineering Principles	21
III	Models	23
9	LightGBM for Tabular Volatility	24
9.1	What Goes In	24
9.2	Configuration	24
9.3	Custom QLIKE Objective	25
9.4	SHAP Interpretability	25
10	LSTM for Intraday Sequences	26
10.1	Architecture	26
10.2	Embedding Alternative	26
10.3	Why Deep Learning Here	27
11	The Ensemble	28
11.1	Architecture	28
11.2	Prediction-Level Blending	29
11.3	Why Two Branches	29
12	Interpretable Trees and Rashomon Analysis	30
12.1	Optimal Decision Trees	30
12.2	Rashomon Analysis Pipeline	31
12.3	What This Tells Us	31
12.4	Evaluation	32
12.5	Novelty	32
IV	Making It Work	33
13	Evaluation	34
13.1	Metrics	34
13.2	Validation Protocol	34
13.2.1	Purged k-Fold CV with Embargo	34
13.2.2	Walk-Forward Evaluation	35
13.3	Success Target	35
14	The Complete Pipeline	36
14.1	End-to-End System Diagram	36
14.2	Implementation Order	36
14.3	Re-training and Monitoring	37
14.4	Lookahead Bias Checklist	37
V	The Build	38
15	The Data-to-Feature Pipeline	39
15.1	Data Lineage	39
15.2	Complete Feature Matrix	39

16 System Architecture	42
16.1 Three Architectures	42
16.2 Feature Stacking	42
16.3 Residual Stacking	43
16.4 Comparison	43
17 Modular Pipeline Design	45
17.1 Pipeline Architecture	45
17.2 Plug Points	45
17.2.1 Feature Layers	45
17.2.2 Models	45
17.2.3 Cross-Validation	46
17.3 Configuration	46
18 The Development Plan	47
18.1 Milestones	47
18.2 Critical Path	47

Part I

The Project

Chapter 1

What We’re Forecasting

Realized variance on day t is $RV_t = \sum_{i=1}^M r_{t,i}^2$, the sum of M squared intraday returns sampled at 5-minute frequency (?). Our target is $\log RV_{t+h}$ for horizons $h \in \{1, 5, 22\}$ trading days, where multi-day targets aggregate: $RV_{t+1:t+h} = h^{-1} \sum_{j=1}^h RV_{t+j}$.

1.1 The Universe

The model is trained and evaluated on 35 instruments over 11.3 years of history (Jan 2012 to Mar 2023).

Table 1.1: Forecast universe: 35 instruments.

Category	Count	Examples
Mega-cap US equities	30	AAPL, MSFT, JPM, JNJ, XOM, AMZN
Broad-market ETFs	4	SPY, QQQ, IWM, DIA
Equity index futures	1	E-mini S&P 500 (ES)
Total	35	

All RV series are computed from 5-minute returns using the realized kernel estimator of ?, which corrects for microstructure noise. Models are fit per-asset (no pooling across names).

1.2 Success Criteria

The primary metric is QLIKE, a quasi-likelihood loss that penalizes relative forecast errors and is robust to noisy volatility proxies (?). The formal definition appears in Chapter 13.

Table 1.2: Evaluation criteria.

Priority	Metric	Requirement
Primary	QLIKE	30–80 bps improvement over HARQ baseline (?), averaged across universe
Secondary	MSE, MAE	Reported alongside QLIKE for robustness
Secondary	Diebold-Mariano test	Statistically significant improvement ($p < 0.05$) vs. each baseline (?)
Tertiary	Economic value	Out-of-sample utility gain in a volatility-targeting portfolio (?)

1.3 The High-Level Pipeline

Figure 1.1 shows the end-to-end forecasting system. Raw data from six sources feeds into a layered feature-engineering pipeline (Layers 0–7, detailed in Chapters 3–8). Two model families consume the resulting feature matrix: LightGBM (Ch. 9) and an LSTM operating on intraday return sequences (Ch. 10). Their forecasts are combined via an ensemble blend (Ch. 11) and evaluated against QLIKE.

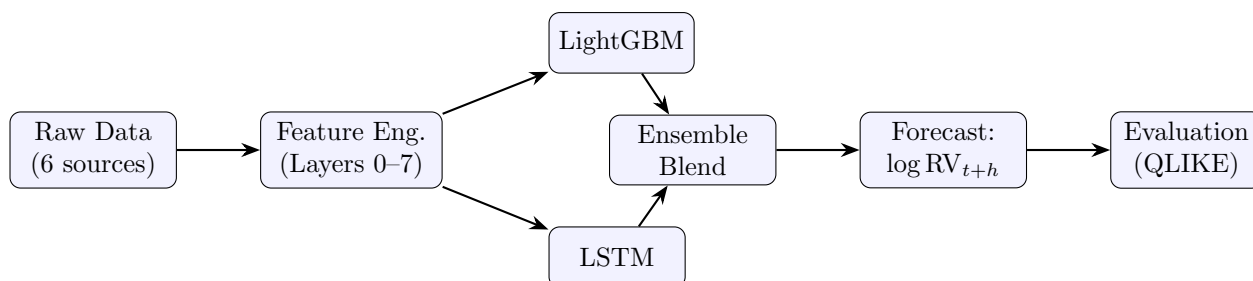


Figure 1.1: High-level forecasting pipeline.

The six raw data sources are: (i) tick-level trade prices, (ii) daily OHLCV, (iii) E-mini S&P 500 Level 2 order book, (iv) SPX implied volatility surface, (v) cross-asset prices (VIX, bonds, gold, oil, FX), and (vi) earnings and macro event calendars.

Features Over Models

The feature set matters more than the model. Layers 0–2 (approximately 20 features covering HAR components, jump decomposition, and signed volatility) achieve 85% of the forecasting accuracy attainable with the full 120-feature set.

Chapter 2

Our Data

2.1 Data Sources

Table 2.1 inventories every data source available for the project. All sources span 11.3 years of history.

Table 2.1: Available data sources, what they enable, and their constraints.

Data Source	Granularity	What It Enables	Key Constraint
Tick-level RV (34 symbols)	L1 tick	RV at any frequency, RQ, jumps, semivariances	—
Daily OHLCV (34 + VIX)	Daily	HAR baselines, ML training	No intraday structure
E-mini L2 depth	L2 (~4M ticks/day)	OBI, depth ratio, VPIN, LSTM input	E-mini only
IV surfaces (Marquee EDRVOL)	Full tenor × strike grid	VRP, skew, term structure, butterfly	Full surface SPX only; single-stock ATM IV via EDRVOL_PERCENT
VIX term structure	Daily	Regime detection, contango/backwardation	—
Cross-asset (treasuries, FX, commodities)	Mixed (tick to daily)	Spillover features, macro regime	Daily for some instruments

2.2 GS Edge vs. Public Data

Table 2.2 summarizes the capabilities our data provides relative to what is typically available in academic studies.

The data advantage is structural, not just bigger

The tick-level RV, full IV surface, and L2 depth data let us construct features that are mathematically impossible to compute from public data. This is not a matter of having “more data”; public researchers literally cannot replicate the feature set.

2.3 Constraints That Shape Decisions

L2 depth data covers the E-mini only, so microstructure depth features (OBI at levels 2–5, depth ratio) apply exclusively to the index. Equities receive L1 features only. The full IV surface is SPX only, so surface-derived features (skew, term structure, butterfly) function as market-wide regime signals. Single-stock ATM IV is available via Marquee EDRVOL_PERCENT, enabling per-name VRP and

Table 2.2: Data advantages over standard academic sources.

Capability	Our Data	Public Alternative	Edge
RV estimation quality	Tick-level, 34 symbols, 11.3 years	5-min returns from TAQ / Oxford-Man	Precise RQ, jump detection, kernel estimators
Options surface	Full SPX tenor $\times$ strike grid (Marquee)	VIX only (model-free 30-day)	Full surface derivatives: skew, butterfly, slope, event-implied
Microstructure depth	E-mini L2 (4M ticks/day)	L1 quotes only (TAQ)	OBI at depth levels 2–5, true depth imbalance
Cross-asset sync	Same tick timestamp across asset classes	Daily closes only	Intraday lead-lag detection
Panel breadth	30 mega-caps + 4 ETFs + E-mini	Typically 1 index or 29 DJIA	Graph models, sector structure, cross-sectional features

IV–RV gap features. These constraints determine which features apply to which assets in the feature engineering pipeline.

Do not treat depth or surface features as stock-level predictors

L2 depth and the full IV surface are index-level data. Using them as if they vary across individual equities would introduce look-ahead bias or a constant signal masquerading as stock-specific information. Single-stock ATM IV is the exception—it is genuinely per-name.

Part II

The Feature Set

Chapter 3

HAR Core and Measurement Quality

The HAR baseline and its noise-aware extension HARQ form Layer 0 of the feature pipeline. Five features, three horizons, one interaction term.

3.1 The Five Foundation Features

Table 3.1: Layer 0 feature set: the five foundation features.

Feature	Definition	Role in Forecast
$\log \text{RV}_t^{(d)}$	$\log \text{RV}_t$	Strongest single predictor; log transform gaussianizes the distribution
$\log \text{RV}_t^{(w)}$	$\log\left(\frac{1}{5} \sum_{i=0}^4 \text{RV}_{t-i}\right)$	Medium-memory component; smooths daily noise
$\log \text{RV}_t^{(m)}$	$\log\left(\frac{1}{22} \sum_{i=0}^{21} \text{RV}_{t-i}\right)$	Long-memory regime anchor
RQ_t	$\frac{M}{3} \sum_{i=1}^M r_{t,i}^4$	Measures how noisy today's RV estimate is
$\sqrt{\text{RQ}_t} \cdot \text{RV}_t^{(d)}$	RQ interaction	Shrinks daily weight on noisy days; highest-impact HAR extension

The baseline HAR model (?) regresses future log-RV on three horizons:

$$\log \text{RV}_{t+1} = \beta_0 + \beta_d \log \text{RV}_t^{(d)} + \beta_w \log \text{RV}_t^{(w)} + \beta_m \log \text{RV}_t^{(m)} + \varepsilon_{t+1}. \quad (3.1)$$

HARQ (?) adds the RQ interaction to make the daily coefficient state-dependent:

$$\log \text{RV}_{t+1} = \beta_0 + (\beta_d + \beta_{dQ} \sqrt{\text{RQ}_t}) \log \text{RV}_t^{(d)} + \beta_w \log \text{RV}_t^{(w)} + \beta_m \log \text{RV}_t^{(m)} + \varepsilon_{t+1}. \quad (3.2)$$

The estimated $\hat{\beta}_{dQ}$ is consistently negative: on noisy days (RQ high), the effective daily coefficient $\beta_d + \beta_{dQ} \sqrt{\text{RQ}_t}$ shrinks toward zero, shifting forecast weight to the weekly and monthly averages.

3.2 The HARQ Shrinkage Mechanism

Figure 3.1 illustrates the core idea: RQ acts as a noise gate on the daily signal.

HARQ with five features consistently beats ML models that use dozens of features without noise-awareness (?). The mechanism is simple: when the daily RV estimate is unreliable, the model automatically falls back to longer-horizon averages that average out the noise. This is the single most robust finding in the HAR literature, replicated across asset classes and sample periods.

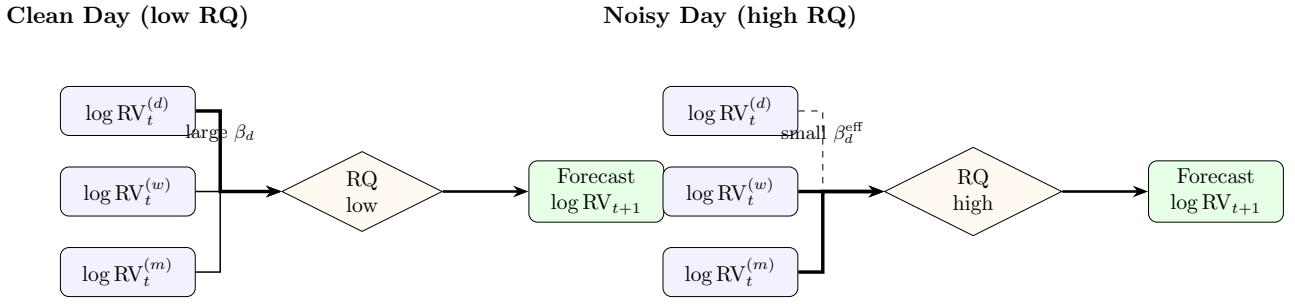


Figure 3.1: HARQ shrinkage mechanism. On clean days the daily reading dominates (thick arrow, left). On noisy days the RQ interaction shrinks the daily coefficient, shifting weight to weekly and monthly averages (thick arrows, right).

3.3 Baseline Performance

These five features alone explain 40–60% of next-day log-RV variation across typical equity and index series. RQ requires tick-level returns; we have them for all 34 symbols in the universe (Table 1.1). This is the non-negotiable core of the feature pipeline. Everything in Chapters 4–7 provides marginal improvement on top of this foundation.

Table 3.2: Expected QLIKE improvement from cumulative feature additions (indicative, based on literature benchmarks).

Layer	Feature Group	Cumulative QLIKE Gain
0	HAR (3 horizons)	Baseline
0	+ RQ interaction (HARQ)	5–15%
1	+ Signed RV, jumps	8–20%
2	+ VRP, IV skew	12–25%
3	+ Microstructure (OBI)	15–30%

Work in Log-RV Space

Raw RV is right-skewed with heavy tails; log-RV is approximately Gaussian (?). This affects loss functions, residual diagnostics, and model comparison. All features in this guide use log-RV unless stated otherwise. When computing QLIKE, convert back to levels: $\widehat{RV}_{t+1} = \exp(\widehat{\log RV}_{t+1} + \hat{\sigma}^2/2)$, where $\hat{\sigma}^2$ is the residual variance (bias correction for the log-normal retransformation).

RQ Interaction: The Single Most Important HAR Extension

Adding the RQ interaction term to baseline HAR yields 5–15% QLIKE improvement from a single feature (?). It is the highest-impact extension before adding any external data. Any ML model that does not account for measurement noise in daily RV leaves this gain on the table.

Chapter 4

Asymmetric Volatility

Layer 1 decomposes realized variance into signed components and jump variation. Six features, all computed from tick-level 5-minute returns.

4.1 Features

Table 4.1: Layer 1 feature set: asymmetric volatility and jump decomposition.

Feature	What It Is	What It Does	Horizon
RS_t^- (daily)	$\sum_{i=1}^M r_{t,i}^2 \mathbf{1}(r_{t,i} < 0)$	Carries $\sim 2\times$ predictive weight of RS^+ ; dominant asymmetric signal (?)	1d
RS_t^+ (daily)	$\sum_{i=1}^M r_{t,i}^2 \mathbf{1}(r_{t,i} \geq 0)$	Weaker predictor; provides the contrast that identifies asymmetry	1d
$RS_t^{-(w)}$ (weekly)	$\frac{1}{5} \sum_{j=0}^4 RS_{t-j}^-$	Persistent downside memory; smooths daily noise in the negative semivariance	1d–5d
J_t^- (signed neg. jump)	Large negative moves beyond threshold	1–3% QLIKE gain beyond unsigned jumps (?)	1d–5d
C_t (continuous variation)	$\max(\text{BPV}_t, 0)$	Highly persistent (ACF ~ 0.6 – 0.7); workhorse of the decomposition (?)	All
J_t (jump variation)	$\max(\text{RV}_t - \text{BPV}_t, 0)$	Nearly unpredictable (ACF ~ 0.0 – 0.1); signals regime breaks	Event

4.2 The Leverage Effect

Negative returns increase future volatility more than positive returns of equal magnitude. The effect is strongest for equity indices and E-mini futures, where RS^- carries roughly twice the predictive weight of RS^+ in HAR-type regressions (?). The SHAR model exploits this directly by replacing RV with separate RS^+ and RS^- regressors at each horizon.

4.3 Continuous vs. Jump Variation

Continuous variation C_t is highly persistent and drives the bulk of out-of-sample forecast accuracy; it behaves like a smoothed version of RV with jump contamination removed. Jump variation J_t is nearly

unpredictable day-to-day, but large positive values flag regime transitions (macro announcements, flash crashes) where the conditional distribution shifts. In practice, C_t is the forecasting workhorse and J_t is the alarm: C enters the model as a persistent regressor at all horizons, while J triggers recalibration or regime-switch logic. The HAR-CJ model of ? formalizes this split and shows that separating C from J yields 1–3% QLIKE improvement over models that use total RV alone.

4.4 Cumulative Performance

Layers 0+1 together (11 features: 5 from HAR/HARQ, 6 from this chapter) achieve approximately 70% of the total attainable QLIKE improvement, per literature benchmarks. All semivariances and jump components are computed from tick-level data at 5-minute frequency. Jump significance is assessed via the Lee-Mykland test, which flags individual intraday returns exceeding a time-varying threshold calibrated to local bipower variation.

Signed Semivariances: The Cheapest Upgrade

Replacing RV with RS^+ and RS^- in a HAR regression (the SHAR model) costs zero additional data and yields 3–8% QLIKE improvement (?). This is the single cheapest feature upgrade after the RQ interaction from Layer 0.

Jumps Are Not Predictors

J_t has near-zero autocorrelation. Do not include lagged jump variation as a standard regressor expecting it to forecast future RV. Its value is as a conditioning variable: when J_t is large, shrink the daily coefficient or trigger a regime indicator. Treating J_t as a regular feature wastes a degree of freedom and can degrade forecast accuracy.

Chapter 5

Options-Implied Features

Layer 2 adds nine features extracted from the SPX implied volatility surface. These are the first forward-looking inputs in the pipeline: they embed market expectations about future variance that backward-looking RV measures cannot capture.

5.1 Features

Table 5.1: Layer 2 feature set: options-implied features from the SPX surface.

Feature	What It Is	Horizon Impact
ATM IV (30-day)	Average of 50-delta put and call IVs from Marquee ERDVOL surface:	All horizons (?)
VRP	$IV_t^{ATM} = \frac{1}{2}(\sigma_{50\Delta P} + \sigma_{50\Delta C})$ $VRP_t = (IV_t^{ATM})^2 - \mathbb{E}_t[RV_{t+1:t+22}]$, estimated using HAR forecast as the RV expectation	1w–1m strongest (?)
25-Delta Risk Reversal	$RR_t = \sigma_{25\Delta C} - \sigma_{25\Delta P}$; measures skew demand for downside protection	1–5d
Term Structure Slope	$TS_t = IV_t^{3m} - IV_t^{1m}$; positive in calm markets, inverts pre-crisis	1w–1m
Butterfly	$BF_t = \sigma_{25\Delta P} + \sigma_{25\Delta C} - 2 IV_t^{ATM}$; measures tail thickness beyond skew	Crisis detection
VVIX	$VVIX_t$: implied volatility of VIX options; vol-of-vol signal	1–5d
VIX Term Structure	$VTSt = F_t^{3m}/VIX_t$; ratio > 1 = contango (calm), < 1 = backwardation (stress)	Regime signal
IV–RV Gap	$Gap_t = IV_t^{ATM} - \sqrt{RV_t^{(m)} \times 252}$; residual risk premium beyond VRP	1–5d
Event-Implied Vol	Extracted from the surface around known event dates (FOMC, earnings); isolates the vol the market attributes to the event	Pre-event only

5.2 Horizon Dependence

Options-implied features contribute almost nothing at $h = 1$ but become the dominant source of marginal improvement at weekly and monthly horizons. Figure 5.1 summarizes the pattern.

The mechanism is straightforward. At $h = 1$, yesterday’s RV is an excellent predictor of tomorrow’s RV because volatility is persistent; options add little incremental information. At $h = 5$ and $h = 22$,

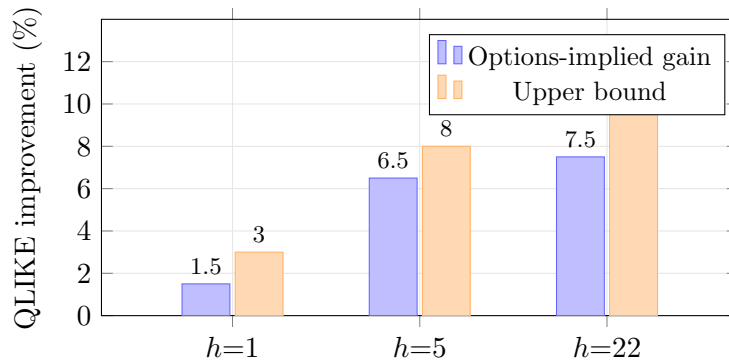


Figure 5.1: QLIKE improvement from adding Layer 2 options features on top of Layers 0–1, by forecast horizon. Ranges: $h=1$: 1–3%; $h=5$: 5–8%; $h=22$: 5–10%. Based on literature benchmarks.

the daily signal decays, and the forward-looking content of options prices fills the gap. Options embed expectations about scheduled events (FOMC, earnings, CPI) that backward-looking RV cannot see.

5.3 What We Compute

The full SPX IV surface is sourced from Marquee `ERDVOL_PERCENT_STANDARD`. This provides a complete tenor $\times$ delta grid from which all nine features are extracted.

The surface covers SPX only. For individual equities, the options-implied features serve as market-wide regime signals (shared across all names in the universe), not stock-specific predictors. This is consistent with the constraint noted in Chapter 2: no single-name IV surfaces are available.

5.4 Cumulative Performance

With Layers 0–2 complete (HAR core, asymmetry/jumps, options-implied), the feature set contains approximately 20 features and captures roughly 85% of the forecasting accuracy achievable with the full 120-feature pipeline. The remaining layers (microstructure, cross-asset, engineered interactions) provide diminishing marginal gains.

ML Advantage Grows with Horizon

At $h = 1$, HAR ties ML on QLIKE. At $h = 5$ and $h = 22$, ML models with options-implied features deliver 10–20% QLIKE improvement over the best linear baseline. The advantage comes from nonlinear interactions between VRP, skew, and term structure that linear models cannot capture (?).

VRP Requires a Realized Variance Forecast

The VRP is defined as $(IV)^2 - \mathbb{E}[RV]$. The second term is itself a model output (typically HAR). Using VRP as an input feature creates a recursive dependency: the VRP feature quality depends on the baseline forecast quality. Use a simple, fixed HAR estimate for the RV expectation term, not the model being trained.

Chapter 6

Microstructure Features

Layer 3 extracts predictive signal from the limit order book and trade flow. These features capture information arrival that daily aggregates cannot see.

6.1 Features

Table 6.1: Layer 3 microstructure features.

Feature	What It Is	Evidence
Price acceleration	$\sum_i (\Delta \log P_i - \Delta \log P_{i-1})^2$	Single most predictive micro feature (Optiver competition)
WAP log returns	Returns computed from volume-weighted average price	Less bid-ask bounce than midprice returns
Order-book imbalance (OBI)	$\frac{\text{bid_size} - \text{ask_size}}{\text{bid_size} + \text{ask_size}}$ at L1–L5	(?)
Depth ratio	$\sum \text{bid depths} / \sum \text{ask depths}$	Structural imbalance across book levels
Market urgency	spread $\times$ OBI	Wide spread + imbalanced book signals imminent move
Spread dynamics	Level, volatility, and momentum of the bid-ask spread	Increasing spread is predictive of near-term vol
Signed volume flow	volume $\times$ sign(trade direction)	Net buying/selling pressure
Sub-window RV ratio	$\text{RV}_{\text{last 5min}} / \text{RV}_{\text{first 5min}}$	Within-window acceleration; captures intraday regime shifts
VPIN	Volume-synced probability of informed trading	(?)

6.2 The Optiver Evidence

The Kaggle Optiver Realized Volatility competition (2021) produced the largest public benchmark for microstructure-driven vol forecasting. LightGBM solutions dominated neural networks by roughly 4:1 on the leaderboard. Price acceleration was the single highest-importance feature across top solutions, and sub-window aggregations (e.g., last-5-minute vs. first-5-minute statistics) consistently outperformed whole-window summaries. A representative top-100 solution (91st place) used approximately 600 features, with each base quantity expanded via {level, change, z-score} engineering.

6.3 Engineering Principle

For each base quantity in Table 6.1, compute three variants:

Table 6.2: Feature triple expansion.

Variant	Definition	Purpose
Level	x_t	Current state
Change	$x_t - x_{t-1}$	Directional momentum
Z-score	$(x_t - \bar{x}_{20})/s_{20}$	Deviation from recent norm

This triples the feature count from 9 base quantities to 27. Tree-based models handle the resulting redundancy naturally via split selection; no manual decorrelation is needed.

6.4 Data Constraints

L2 order-book depth (five levels of bid/ask size) is available only for the E-mini S&P 500 (ES) via SecDB. Equities receive L1 features only: spread, OBI at best bid/ask, and signed volume flow. Features requiring multi-level depth (depth ratio, market urgency with L2–L5 OBI) are ES-only.

The raw intraday sequences that produce these features also feed the LSTM module (Ch. 10), which consumes them as time-ordered input rather than daily aggregates.

Lookahead Bias in Microstructure Features

Features for RV_{t+1} must use only information available at the close of period t . Timestamp alignment is critical: if RV_{t+1} covers trading day $t + 1$, then features must be computed from data up to and including the close of day t . A single misaligned sub-window statistic will inflate backtest performance and produce unreproducible results in production.

Microstructure Features Add Most Value for Trees

Microstructure features are high-cardinality, noisy, and partially redundant. LightGBM exploits them effectively via greedy splits; linear models and shallow networks gain less. The Optiver evidence confirms this: tree ensembles dominated when the feature set was microstructure-heavy.

Chapter 7

Cross-Asset Spillovers

Cross-asset features form Layer 4 of the feature pipeline. They capture vol transmission across asset classes, adding 1–5% QLIKE improvement concentrated in regime transitions.

7.1 Features

Table 7.1: Layer 4 feature set: cross-asset spillover signals.

Feature	What It Is	Mechanism
Treasury slope change	$\Delta(10y - 2y \text{ yield})$	Rate curve inversion precedes equity vol spikes by days
Credit spread momentum	$\Delta \text{ IG/HY spread or TY futures vol}$	Credit stress leads equity vol
FX vol (USD/JPY)	RV of USD/JPY	Yen carry unwind = global risk-off = equity vol spike
Commodity vol (CL, GC)	RV of crude oil + gold	Oil = macro uncertainty; gold = flight-to-safety intensity
DY Spillover Index	VAR(5) variance decomposition on 34-asset RV panel	Fraction of vol driven by cross-asset contagion; spikes in crises (?)
Sector-mean RV	Average RV across same-sector names	Filters idiosyncratic noise
VIX-equity corr. regime	Rolling 20-day $\text{corr}(\Delta \text{VIX}, \text{SPX returns})$	Near -1 normally; breaks during regime shifts
Cross-asset RV rank	Each asset's RV percentile relative to peers	Detects outlier dispersion regimes

7.2 Impact

Cross-asset features contribute 1–5% QLIKE improvement on average, but the gains concentrate in regime transitions where forecasts are most valuable and hardest to get right. The improvement is small in calm markets and large in the tails: exactly when single-asset models break down, cross-asset signals provide early warning.

7.3 Graph-HAR

Graph-HAR extends the standard HAR by adding a neighbor-weighted RV term:

$$\log \text{RV}_{i,t+1} = \beta_0 + \beta_d \log \text{RV}_{i,t}^{(d)} + \beta_w \log \text{RV}_{i,t}^{(w)} + \beta_m \log \text{RV}_{i,t}^{(m)} + \gamma \sum_{k \neq i} W_{ik} \log \text{RV}_{k,t} + \varepsilon_{i,t+1}. \quad (7.1)$$

The term $\gamma \sum_k W_{ik} \log RV_{k,t}$ captures how asset i 's vol tomorrow depends partly on its neighbors' vol today. For example, AAPL's forecast improves by incorporating MSFT's realized vol when the two are tightly connected. The weight matrix W can be correlation-based (simple, static) or learned via a graph neural network (?).

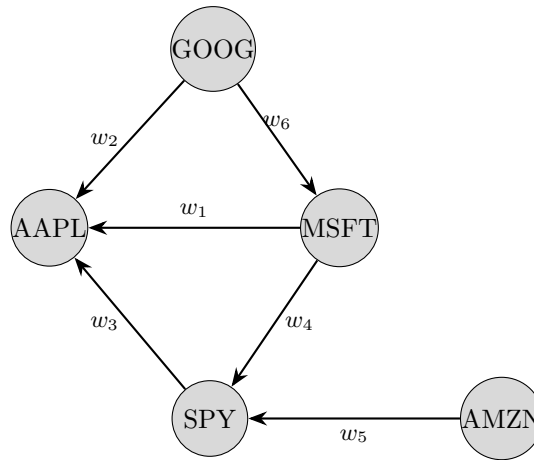


Figure 7.1: Cross-asset vol network. Each node is an asset; directed edges carry weights w_k determining how much neighbor RV feeds into a given asset's forecast. Weights can be fixed (correlation) or learned (GNN).

DY Spillover Index as a Regime Detector

The ? total spillover index measures the fraction of forecast-error variance attributable to cross-asset shocks. It runs at 30–40% in calm markets and spikes above 70% in crises (2008, 2020). As a feature, its level signals contagion intensity; its rate of change signals regime entry/exit.

Nonsynchronous Trading Across Asset Classes

Bonds, FX, and commodities trade on different schedules than equities. Naive daily alignment introduces stale-price bias that inflates apparent lead-lag relationships. Use overlap-window RV (common trading hours only) or the Hayashi-Yoshida estimator for cross-asset covariance.

7.4 GS Advantage

Synchronized tick data across asset classes (equities, futures, FX, treasuries) enables intraday lead-lag detection at minute-level granularity. Academic papers rely on daily closes, making sub-day cross-asset transmission invisible and leaving the strongest spillover signals unexploited.

Chapter 8

Feature Composition and Selection

This chapter synthesizes Layers 0–4 (Chapters 3–7), adds the remaining layers (calendar, memory, sentiment), and provides the decision framework for which features to use at each forecast horizon.

8.1 Calendar, Memory, and Sentiment (Layers 5–7)

Layers 5–7 are individually weak but collectively additive. They are the last 5% of achievable accuracy.

Layer 5: Calendar Features

Table 8.1: Calendar features and when they matter.

Feature	When It Matters
FOMC indicator (−1, 0, +1, +2 days)	Vol compression before announcement, expansion after
NFP/CPI indicator	Macro release effect on intraday and next-day vol
Options expiry (monthly/quarterly)	Pinning and gamma unwind
Quarter-end rebalancing	Last 3 days of quarter; forced portfolio flows
Earnings proximity	Days to next earnings release (single names only)
Time-of-day	U-shape: open and close have highest vol
Day-of-week	Weakening over time but still detectable

Layer 6: Memory Features

Table 8.2: Memory features capturing long-range dependence and regime state.

Feature	What It Is
Fractionally differenced RV	$(1 - L)^d$ RV with $d \approx 0.35$ – 0.45 ; preserves long memory while stationary (?)
Rolling Hurst exponent	$H < 0.15$ = rough/fast mean-reversion; $H > 0.3$ = trending (?)
Vol-of-vol	std(RV) over last 22 days; measures instability of the vol process
Regime duration	Days since last 2σ RV spike; acts as a mean-reversion clock

Layer 7: Sentiment

FinBERT news sentiment and negative news count provide 1–3% QLIKE improvement, concentrated in crisis periods (?). Include these features only if the data pipeline effort is justified by the target

asset class. For index-level forecasts (SPX, SPY), sentiment adds less value than for single names with idiosyncratic news flow.

8.2 The Diminishing Returns Curve

Figure 8.1 shows the cumulative accuracy contribution by layer. The first 20 features (Layers 0–2) achieve 85% of attainable accuracy. The remaining 60–100 features add 15%.

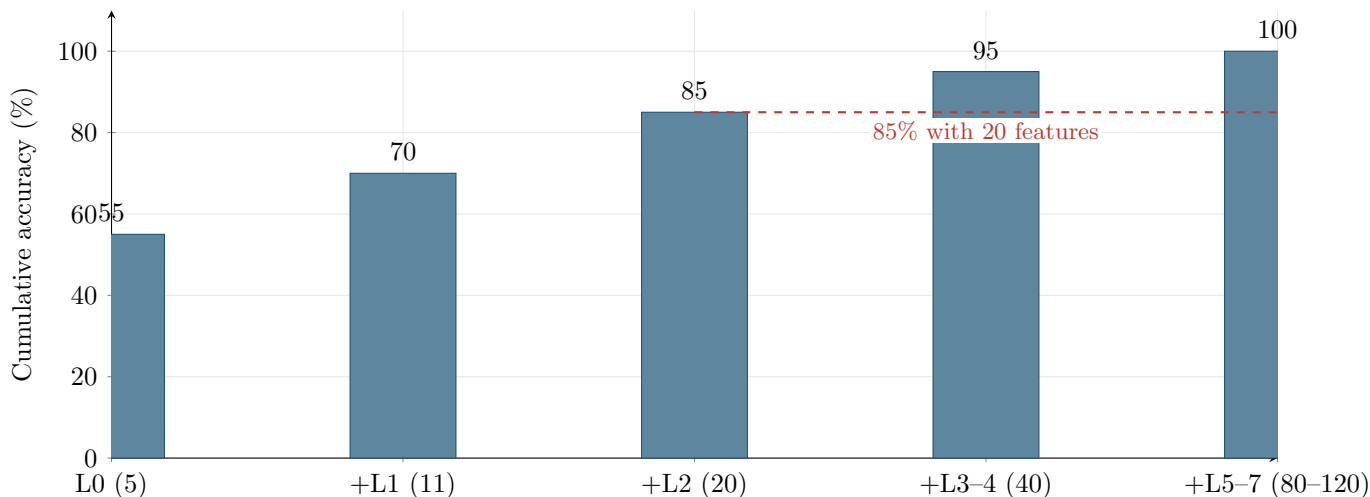


Figure 8.1: Cumulative accuracy by feature layer. Parentheses show total feature count at each stage. The dashed line marks the 85% threshold achievable with Layers 0–2 and a Ridge regression.

8.3 Feature Priority by Forecast Horizon

Table 8.3: Dominant features and ML value-add by forecast horizon.

Horizon	Dominant Features	Where ML Adds Value
Intraday (10min–1hr)	Microstructure (L3): price acceleration, OBI, spread	Trees with 600+ features
1 day	HARQ core (L0) + RQ + asymmetry (L1)	HARQ nearly optimal; ML adds ~5%
1 week	Options (L2) + cross-asset (L4)	VRP + skew: 5–10% over pure RV models
1 month	VRP (L2) + macro (L4) + Hurst (L6)	Options have max informational advantage

The 1-Day Horizon Is the Hardest to Beat

At the 1-day horizon, HARQ with 5 features is nearly optimal. ML models gain ~5% QLIKE improvement at best, and much of that comes from the RQ interaction that HARQ already captures. The largest ML gains are at intraday and weekly+ horizons where linear models lack the capacity to exploit high-dimensional feature sets.

8.4 Feature Engineering Principles

- **Triple expansion.** For each base quantity, compute {level, change, z -score} systematically. This triples the feature count and captures state, direction, and unusualness in a single pass.
- **Horizon-dependent selection.** Drop microstructure features at the monthly horizon (noise at

that scale). Drop calendar features at intraday (FOMC day is already known by 9:30am). Match the feature set to the prediction target.

- **Trees handle redundancy naturally.** Correlated features split across nodes without the multicollinearity problems that plague linear models. No need to pre-decorrelate or drop correlated pairs before training a gradient-boosted ensemble.
- **Feature importance from a single model is unstable.** When features are redundant, a single LightGBM run produces noisy importance rankings that shift across random seeds. Rashomon analysis (Chapter 12) addresses this by examining the set of near-optimal models rather than one point estimate.

Features Over Models

The feature set matters more than the model. Layers 0–2 achieve 85% of attainable accuracy with 20 features and a Ridge regression. The remaining 15% requires 60–100 more features and careful model architecture. Invest in feature engineering first; optimize the model second.

Part III

Models

Chapter 9

LightGBM for Tabular Volatility

LightGBM (?) is the primary ML model in the pipeline. It ingests the full engineered feature set from Layers 0–7 and optimizes a custom QLIKE objective.

9.1 What Goes In

Table 9.1 summarizes the raw feature counts per layer before expansion. After applying {level, change, z-score} transformations to each base feature, the final input dimensionality is approximately 80–120.

Table 9.1: Feature layers feeding the LightGBM model.

Layer	Features	Count
0	log RV (d/w/m), RQ, RQ interaction	5
1	RV^+ , RV^- , signed jumps, C , J	6
2	ATM IV, VRP, skew, term structure, VVIX, butterfly, IV–RV gap, stock IV, stock VRP	9
3	Price acceleration, OBI, spread, VPIN, Kyle λ	9
4	Treasury slope, FX vol, commodity vol, VIX, VIX futures slope, DY spillover	6
5	Calendar: FOMC, OpEx, month/weekday dummies	~15
6	Memory: Hurst exponent, fractional d , ACF features	~10
7	Sentiment: FinBERT scores, attention proxies	~15–55
Raw total		~37–57
After {level, change, z-score} expansion		~80–120

9.2 Configuration

Table 9.2 gives the reference configuration, adapted from the Optiver Kaggle competition (91st-percentile solution).

Table 9.2: LightGBM reference configuration.

Parameter	Value	Notes
learning_rate	0.05	
num_leaves	255	
min_data_in_leaf	255	Prevents overfitting on rare regimes
n_estimators	10,000	With early stopping
early_stopping_rounds	400	
boosting_type	dart	Dropout regularization
objective	Custom	QLIKE loss; not a built-in objective
metric	Custom	QLIKE evaluation for early stopping

9.3 Custom QLIKE Objective

QLIKE is not a built-in LightGBM loss. Training requires a custom objective function that returns the gradient and Hessian of the QLIKE loss with respect to the predicted value, plus a separate custom evaluation metric for early stopping. QLIKE is robust to noise in the volatility proxy: it ranks forecasts consistently even when RV is measured with error, unlike MSE (?). All training and prediction operate in log-RV space; convert back to levels only for final QLIKE evaluation.

9.4 SHAP Interpretability

SHAP values (?) provide feature importance and interaction effects for each prediction. They are required for the GS presentation and for defending model behavior under scrutiny. However, single-model importance measures (gain, permutation, SHAP) are unstable across refits when features are near-substitutes (e.g., VIX, VVIX, ATM IV all proxy for the same latent factor). This instability motivates the Rashomon analysis in Chapter 12, which quantifies importance across the set of near-optimal models rather than relying on a single fit.

Baseline First

The fitting scheme for the HAR baseline matters more than the choice of ML model (?). A properly fitted HAR using OLS with Newey–West standard errors and an expanding or rolling window is essential before claiming ML improvement. Without this baseline, apparent ML gains may reflect a weak benchmark, not genuine forecasting ability.

Why LightGBM

Gradient-boosted trees dominate tabular volatility forecasting benchmarks (??). They handle mixed feature types (continuous RV lags, categorical calendar dummies, ordinal sentiment scores) without preprocessing, capture nonlinear interactions automatically, and train in seconds on datasets of this size ($\sim 5,000$ rows $\times$ 120 features).

Chapter 10

LSTM for Intraday Sequences

The second model branch operates on sequential intraday data rather than daily tabular features. E-mini S&P 500 Level 2 tick data ($\sim 4\text{M}$ ticks/day) is compressed into 5-minute (or 1-minute) return bars, each augmented with a limit-order-book snapshot. One training sample is one full trading day's bar sequence. The model's sole task is to produce an independent next-day log RV forecast from this intraday structure.

10.1 Architecture

Figure 10.1 shows the data flow from raw ticks to the ensemble blend in Ch. 11.

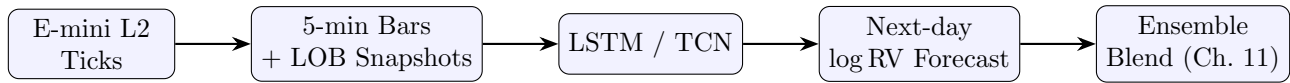


Figure 10.1: Intraday sequence model pipeline.

1. A small LSTM (or TCN) consumes the intraday E-mini sequence for each trading day.
2. Input per time step: 5-min log-return plus LOB snapshot features (bid-ask spread, depth imbalance, trade imbalance, mid-price change).
3. Output: a single scalar, the next-day log RV forecast, produced from the final hidden state via a linear head.
4. This forecast is blended with LightGBM's forecast at the prediction level (Ch. 11).

Table 10.1: Baseline sequence model specifications.

Component	LSTM variant	TCN variant
Layers	2	4 dilated causal conv blocks
Hidden dim	64–128	64 channels
Sequence length	78 bars (5-min, full day)	78 bars
Dropout	0.2	0.2 (spatial)
Loss	QLIKE on log RV	QLIKE on log RV
Optimizer	AdamW, cosine LR	AdamW, cosine LR

10.2 Embedding Alternative

Instead of producing an independent forecast, the LSTM can export its last-layer hidden state as a fixed-length embedding vector and feed it into LightGBM as additional tabular features (feature-level

stacking). Competition evidence from Optiver (?) favors prediction-level blending over feature-level stacking, but both approaches should be compared on our data. The embedding path also risks gradient isolation: LightGBM cannot back-propagate into the LSTM, so the embedding is not trained end-to-end for the tabular objective.

10.3 Why Deep Learning Here

Four million ticks per day is too rich for hand-engineered aggregations alone to capture fully. Temporal order within the day carries signal: acceleration patterns in returns, depth shifts preceding volatility spikes, and intraday momentum reversals are sequential phenomena that trees cannot exploit from summary statistics. This is the one place in the pipeline where deep learning genuinely adds value over gradient-boosted trees. Optiver competitors worked with 10-minute windows (short sequences where hand-crafted features captured most of the information); our full-day sequences are substantially richer and harder to summarize manually.

Diminishing Returns

The LSTM branch will almost certainly contribute less marginal accuracy than the top 20 tabular features in LightGBM. Its value is in capturing residual sequential patterns that tabular aggregations miss, not in replacing the tabular model. Invest proportional effort: get the tabular pipeline right first.

DL for Sequences, Trees for Tables

The LSTM operates on sequential intraday data where temporal order matters. LightGBM operates on tabular daily features. Each model gets the data format it handles best. They combine at the prediction level (Ch. 11).

Chapter 11

The Ensemble

The pipeline has two model branches: LightGBM (Chapter 9) consuming 80–120 tabular features from Layers 0–7, and an LSTM/TCN (Chapter 10) consuming intraday E-mini L2 sequences. Each branch produces an independent $\log \text{RV}_{t+h}$ forecast. These forecasts are combined at the prediction level into a single blended output.

11.1 Architecture

Figure 11.1 shows the full two-branch architecture converging into the ensemble blend.

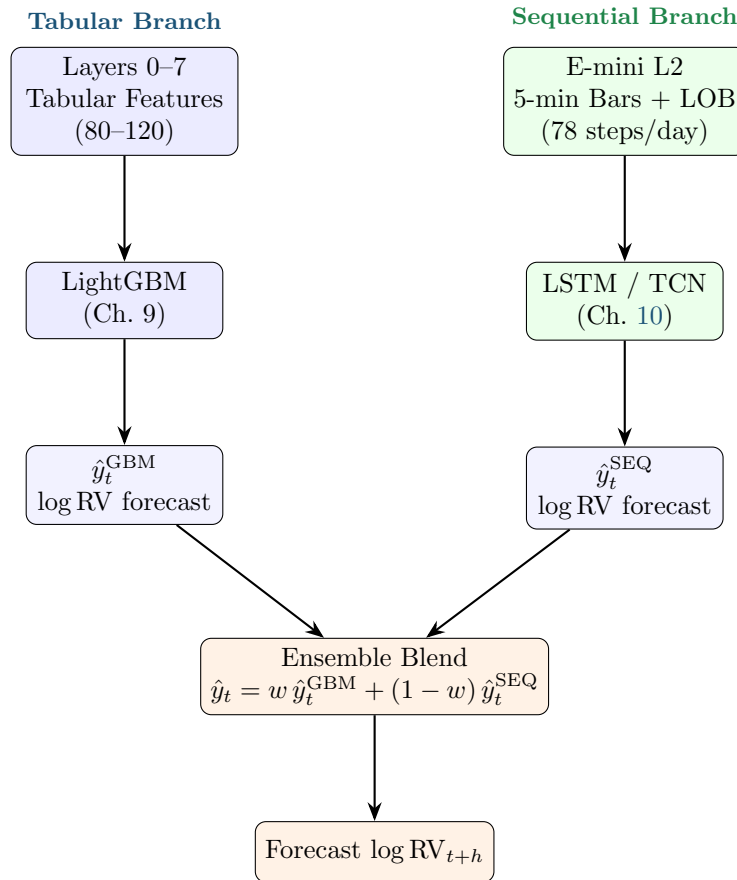


Figure 11.1: Two-branch ensemble architecture. Each branch independently forecasts $\log \text{RV}_{t+h}$; outputs are blended at the prediction level.

11.2 Prediction-Level Blending

We blend model *outputs*, not internal representations. Feature-level stacking (feeding LSTM embeddings as LightGBM inputs) breaks gradient isolation and couples model debugging; top-performing Optiver and AmEx Kaggle solutions consistently found that blending independent predictions outperforms feature-level fusion (?). Blend weights can be static (equal weighting, or a fixed ratio tuned to minimize QLIKE on the validation set) or dynamic (regime-dependent, shifting weight toward the sequential branch during high-microstructure-activity periods). Start with simple validation-tuned static weights; add regime conditioning only if it improves out-of-sample QLIKE.

Do Not Stack Features

Feeding the LSTM's hidden-state embedding into LightGBM as extra columns is tempting but counterproductive. LightGBM cannot back-propagate into the LSTM, so the embedding is never optimized for the tabular objective. Debugging also becomes harder: a degradation in the blend could originate in either model or in the coupling layer. Keep the branches independent.

11.3 Why Two Branches

LightGBM dominates tabular volatility benchmarks (Chapters 3–8 features). LSTM/TCN captures sequential intraday microstructure that tabular summary statistics cannot represent. Giving each model the data format it handles best, then combining, extracts more signal than forcing either model to do both jobs.

Blend Predictions, Not Features

Each model gets the data format it handles best: trees for tabular, recurrent/convolutional networks for sequences. Combining their independent forecasts at the prediction level outperforms feeding one model's internals into the other.

Chapter 12

Interpretable Trees and Rashomon Analysis

LightGBM is a black box. For the GS presentation, we need an interpretable model that can be inspected and defended. Optimal decision trees provide this, and Rashomon analysis reveals which features are genuinely important across the set of near-optimal models.

12.1 Optimal Decision Trees

We train a `STreeDPiecewiseLinearRegressor` from `pystreed` on the same Layers 0–7 features used by LightGBM (Chapter 9). Table 12.1 specifies the configuration.

Table 12.1: Optimal decision tree configuration.

Parameter	Value	Notes
Solver	STreeD	DP with elastic-net leaves
Max depth	4–5	Tuned via purged blocked k -fold CV
Leaf count	8–32	Cost-complexity regularization controls this
Leaf model	Elastic net	Ridge + lasso per leaf partition
Cost-complexity	CV-tuned	Penalizes $ \text{leaves}(T) $
Features	Layers 0–7	Same panel as LightGBM
Target	$\log \text{RV}_{t+h}$	$h \in \{1, 5, 22\}$

Each leaf fits a local elastic-net regression on the features routed to it, so the tree is piecewise-linear rather than piecewise-constant. This gives better accuracy than a constant-leaf tree while remaining fully inspectable: every prediction traces to one root-to-leaf path plus a short linear formula.

Expected accuracy relative to benchmarks:

- ~2–5% higher MSE than tuned LightGBM (acceptable for interpretability).
- ~10% lower MSE than HAR baseline.
- Comfortably inside the Model Confidence Set when evaluated by QLIKE.

Binarization Trap

GOSDT and the original TreeFARMS require binary features. Binarizing continuous vol features (e.g., 50 threshold candidates per feature $\times$ 40 features = 2,000 binary columns) destroys information and inflates runtime exponentially. Use `STreeDPiecewiseLinearRegressor`, which splits on continuous features natively.

12.2 Rashomon Analysis Pipeline

The Rashomon set is the collection of all models whose loss is within ϵ of the optimum. For decision trees, this set can be enumerated exactly (?). Figure 12.1 shows the four-step pipeline.

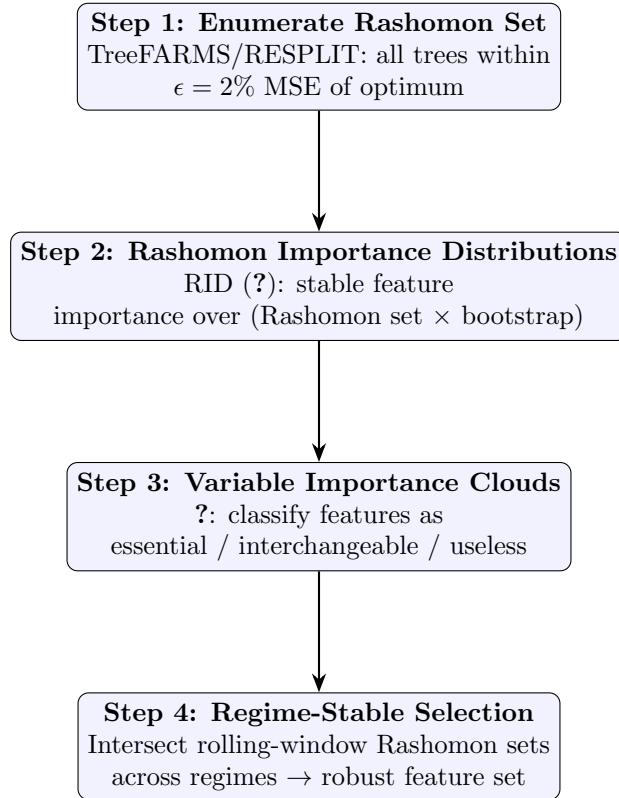


Figure 12.1: Rashomon analysis pipeline. Each step narrows the set of defensible models and the set of robustly important features.

Step 1. TreeFARMS (?) enumerates every sparse decision tree within $\epsilon = 2\%$ MSE of the optimal tree. For the vol feature panel (~ 40 base features, $\sim 5,000$ rows, depth 4–5), expect 10^3 – 10^5 trees in the Rashomon set. RESPLIT (or SPLIT) provides a faster alternative if enumeration is slow.

Step 2. RID (?) computes a stable importance distribution for each feature across the entire Rashomon set, bootstrapped for confidence intervals. Unlike single-model SHAP or permutation importance, RID has consistency theorems and finite-sample error rates.

Step 3. Variable Importance Clouds (?) map each feature to its $[\min, \max]$ importance range across the Rashomon set. Non-overlapping clouds indicate robustly distinct features; overlapping clouds indicate substitutes.

Step 4. Train Rashomon sets on rolling windows (e.g., 5-year training, 1-year step). Intersect the sets across windows: features appearing in *every* regime’s Rashomon set are regime-stable; features appearing in only one regime are fragile.

12.3 What This Tells Us

- **Genuine importance vs. accidental selection.** Single-model importance (gain, permutation, SHAP) is unstable when features are near-substitutes. RID and VIC identify which features are robustly important across all defensible models.
- **Feature substitution structure.** VIX, VVIX, ATM IV, and the IV–RV spread are near-collinear proxies for the same latent factor. Rashomon analysis reveals which are essential (appear in every near-optimal tree), which are interchangeable (substitutable without accuracy

loss), and which are useless (in no near-optimal tree).

- **Prediction multiplicity.** For any input, the range of forecasts across all Rashomon-set trees quantifies prediction uncertainty due to model choice alone. Report the [5th, 95th] percentile forecast range alongside the point forecast for risk reporting.
- **Post-hoc constraint satisfaction.** Filter the Rashomon set for trees that satisfy additional constraints (monotone in VIX, exclude flagged features, ≤ 12 leaves) without retraining. If a compliant tree exists in the set, use it directly.

12.4 Evaluation

Table 12.2 defines the evaluation protocol for the interpretable tree and the Rashomon analysis.

Table 12.2: Evaluation protocol for interpretable trees and Rashomon analysis.

Component	Protocol
Accuracy metrics	Walk-forward MSE, QLIKE, MAE on $\log RV_{t+1}$
Baselines	HAR, HARQ, LightGBM (Chapter 9)
Statistical tests	Diebold–Mariano (?) pairwise tests; Model Confidence Set
Acceptable gap	Optimal tree within 5% MSE of LightGBM; must beat HAR by $\geq 8\%$
Regime stress test	Rashomon prediction range during Mar 2020, Volmageddon (Feb 2018); report [5, 95] percentile spread
Multiplicity audit	Fraction of test days where Rashomon-set forecast range spans >1 QLIKE unit
Feature stability	Jaccard similarity of top-10 RID features across rolling windows

The interpretable tree’s accuracy trade-off is acceptable if: (1) it remains in the Model Confidence Set alongside LightGBM, and (2) the Rashomon prediction range narrows after regime-stable feature selection, confirming that the selected features reduce model ambiguity.

12.5 Novelty

No published paper has applied Rashomon methods to any financial time-series problem. The closest applications are cross-sectional credit risk (FICO HELOC dataset) and criminal justice (COMPAS recidivism). Applying Rashomon-set enumeration, RID, and Variable Importance Clouds to realized volatility forecasting is the novel research contribution of this project.

The Novel Contribution

Rashomon analysis answers: are my features robustly important, or did my model just pick one out of several interchangeable options? This is the novel research contribution.

Part IV

Making It Work

Chapter 13

Evaluation

This chapter defines how forecasts are scored, how models are compared, and what “better” means quantitatively.

13.1 Metrics

Table 13.1 lists every metric used in the project and its role.

Table 13.1: Forecast evaluation metrics.		
Metric	What It Measures	Role
QLIKE	Quasi-likelihood loss; penalizes relative forecast errors	Primary
MSE	Mean squared error on log-RV	Secondary
MAE	Mean absolute error on log-RV	Robustness check
Diebold–Mariano test	Statistical significance of pairwise forecast differences	“Is model A better than B?”
Model Confidence Set	Set of models not significantly worse than the best	“Which models are top tier?”

The primary loss function is QLIKE:

$$\text{QLIKE} = \frac{1}{T} \sum_{t=1}^T \left(\frac{\hat{\sigma}_t^2}{\sigma_t^2} - \log \frac{\hat{\sigma}_t^2}{\sigma_t^2} - 1 \right) \quad (13.1)$$

QLIKE penalizes relative errors, not absolute ones. It ranks forecasters consistently even when RV is measured with noise, because it belongs to the class of loss functions that are robust to imperfect volatility proxies (?).

QLIKE Is the Only Loss That Matters for Ranking

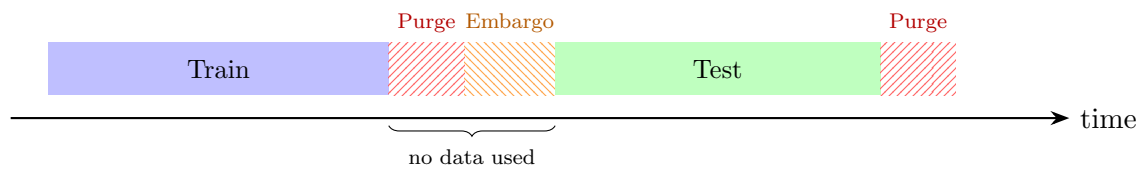
QLIKE is the primary metric for all model selection and comparison decisions. MSE and MAE serve only as secondary diagnostics. The Diebold–Mariano test (?) confirms whether a QLIKE difference is statistically significant; the Model Confidence Set (?) identifies which models belong to the top tier.

13.2 Validation Protocol

13.2.1 Purged k-Fold CV with Embargo

Standard cross-validation leaks information in time series because observations near fold boundaries share overlapping feature windows. Purged CV removes observations within the purge window on

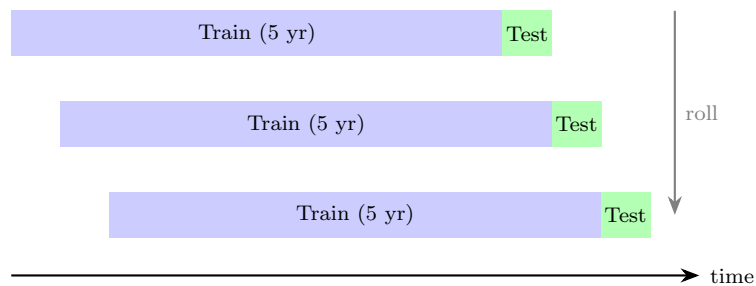
both sides of the train/test boundary. An embargo adds a further gap after each test fold to prevent label leakage from multi-day targets (?).



We use purged 5-fold CV for hyperparameter tuning. Purge window: equal to the longest feature lookback (22 trading days for the monthly HAR component). Embargo window: equal to the forecast horizon (1 or 5 days).

13.2.2 Walk-Forward Evaluation

Walk-forward is the primary out-of-sample evaluation method. Train on a rolling 5-year window, forecast the next period, step forward by one period, and repeat. All reported QLIKE numbers come from this procedure, never from in-sample or CV scores.



13.3 Success Target

Three criteria define success:

1. **QLIKE improvement:** 30–80 bps over the HARQ baseline (?), averaged across the 35-instrument universe.
2. **Statistical significance:** Diebold–Mariano $p < 0.05$ vs. each baseline (HAR, HARQ, SHAR, Realized GARCH).
3. **Economic value:** Positive out-of-sample utility gain in a volatility-targeting portfolio (?).

Train with QLIKE, Not MSE

MSE is dominated by extreme volatility days. A single crisis observation can flip MSE rankings. QLIKE penalizes relative forecast errors and ranks models consistently regardless of the volatility regime (?).

LightGBM does not provide a built-in QLIKE loss. Training requires a custom objective that returns both the gradient and Hessian of the QLIKE loss with respect to the predicted value (see Chapter 9, Section 9.3). If you train with MSE and evaluate with QLIKE, the model optimizes the wrong surface and rankings will not transfer.

Chapter 14

The Complete Pipeline

This chapter assembles the data sources (Chapter 2), feature layers (Chapters 3–8), models (Chapters 9–12), and evaluation framework (Chapter 13) into the end-to-end system and defines the implementation roadmap.

14.1 End-to-End System Diagram

Figure 14.1 shows every component from raw data to scored forecasts.

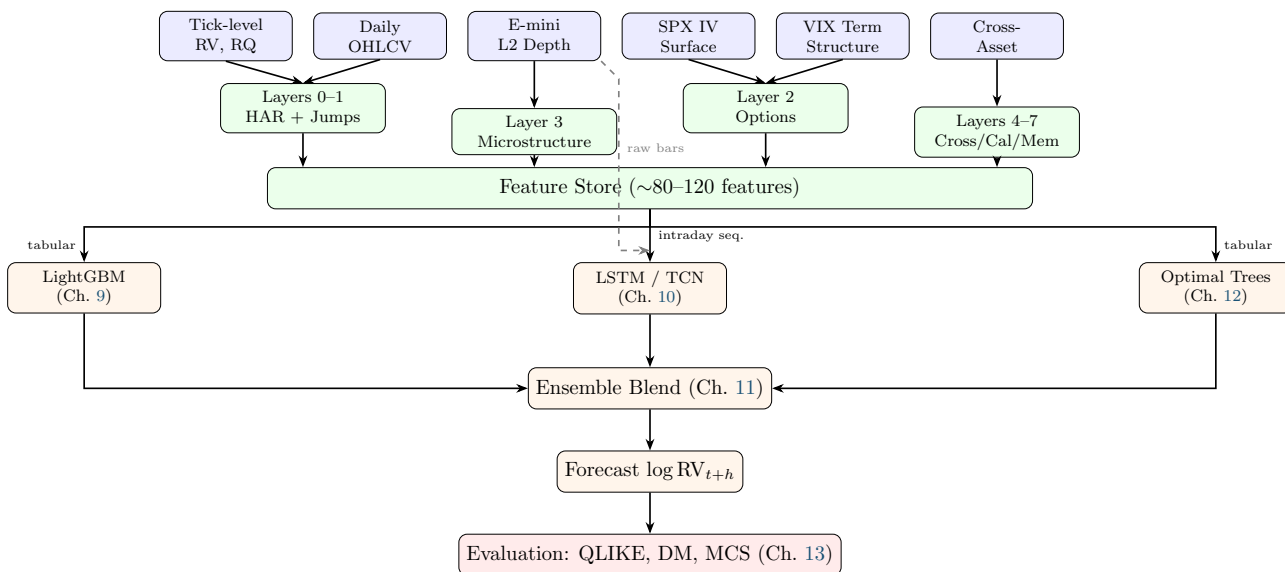


Figure 14.1: End-to-end pipeline from raw data to scored forecasts. Blue: data sources. Green: feature computation and storage. Orange: models and blending. Red: evaluation. Dashed arrow indicates the LSTM also receives raw intraday bar sequences directly.

14.2 Implementation Order

Each step produces a standalone, reportable result. No step depends on a later step being complete.

Steps 1–2 Are the Critical Path

If Step 1 (HARQ/SHAR baseline) is weak, every subsequent QLIKE comparison is misleading (?). Step 2 (adding options features via LightGBM) produces the first genuine ML-vs-baseline comparison. Getting these two steps right matters more than reaching Step 5.

Table 14.1: Implementation roadmap. Each step is independently reportable.

Step	What	Features	Model	Deliverable
1	HARQ + SHAR baseline	Layers 0–1 (11 features)	OLS / Ridge	Walk-forward QLIKE; baseline table
2	Add options layer	Layer 2 (~20 total)	LightGBM	QLIKE lift vs. Step 1; SHAP summary
3	Cross-asset + spillover	Layer 4 (~30 total)	LightGBM	Spillover contribution analysis
4	E-mini microstructure	Layer 3 (separate)	LSTM / TCN	Standalone intraday forecast
5	Full feature set	Layers 5–7 (~80–120)	Ensemble	Final QLIKE; DM tests; MCS
6	Rashomon analysis	Same as Step 5	Optimal Trees	Variable importance stability report

14.3 Re-training and Monitoring

Retrain all models weekly on a rolling 5-year window. After each retrain, compute the Rashomon set and record the Jaccard similarity of the top-10 features compared to the previous window; a drop below 0.6 signals feature-importance drift and warrants investigation. Monitor out-of-sample QLIKE on a trailing 20-day basis; alert if it degrades more than 10% relative to the trailing 60-day average. Log every retrain’s feature ranking and QLIKE for post-hoc regime analysis.

14.4 Lookahead Bias Checklist

Table 14.2 lists the four primary sources of lookahead bias in volatility forecasting systems.

Table 14.2: Lookahead bias sources and prevention rules.

Source	Pitfall	Rule
Realized measures	Intraday returns from the target day leak into features	Features for RV_{t+1} use only information $\leq t$
Microstructure	Full-day LOB features include the close	Truncate intraday sequences at $t - \epsilon$; align timestamps strictly
Options surface	Intraday surface changes reflect information about the target day	Use end-of-day surface from day t for the day- $t+1$ prediction
Cross-asset	Mixed frequencies across asset classes	Synchronize all inputs to the same end-of-day cutoff

Lookahead Bias

The single most common error in financial ML research. Every feature must be computable strictly before the forecast target period begins. When in doubt, add a one-day lag. A lookahead-contaminated model will show excellent in-sample QLIKE that vanishes out of sample, wasting weeks of development time before the bug is identified.

Part V

The Build

Chapter 15

The Data-to-Feature Pipeline

Every feature in the model traces back to one of the six raw data sources in Chapter 2. This chapter maps that lineage: source to measure to feature. For measure formulas, see Chapters 3–7. For selection rationale, see Chapter 8.

15.1 Data Lineage

Figure 15.1 shows how data narrows and expands through the pipeline. Six raw sources produce approximately 18 daily scalar measures. These measures pass through lagging, rolling-window aggregation, and triple expansion to produce the final 80–120 feature matrix.

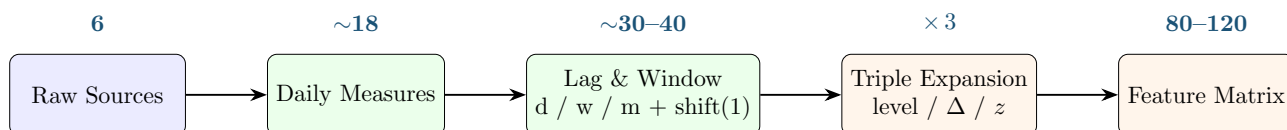


Figure 15.1: Data lineage funnel. Blue: raw inputs (Ch. 2). Green: intermediate measures (Chs. 3–7). Orange: model-ready features. Counts show approximate dimensionality at each stage.

15.2 Complete Feature Matrix

Table 15.1 lists every feature in the planned feature set. Read across any row to trace a feature from its source measure through to its final form. Features with d/w/m variants share the same derivation pattern and are collapsed into one row.

Table 15.1: Complete feature matrix with source lineage. Features marked d/w/m apply the same derivation at daily, weekly (5d), and monthly (22d) horizons.

Layer	Feature	Source Measure	Derivation	Expansion
0	log_rv_d/w/m	rv	log [+ rolling mean 5d/22d], shift(1)	—
0	sqrt_rq_d	rq	$\sqrt{\cdot}$, shift(1)	—
0	rq_rv_interaction	rq, rv	$\sqrt{rq} \cdot \log(rv)$, shift(1)	—
0	overnight_return	open, close (TSDB)	$\log(open_t/close_{t-1})$, shift(1)	—
1	log_rs_positive_d/w/m	rs_positive	log [+ rolling mean 5d/22d], shift(1)	—
1	log_rs_negative_d/w/m	rs_negative	log [+ rolling mean 5d/22d], shift(1)	—
1	log_bpv_d/w	bpv	log [+ rolling mean 5d], shift(1)	—
1	log_jump_variation_d	jump_variation	log, shift(1)	—
1	log_continuous_var_d/w	continuous_variation	log [+ rolling mean 5d], shift(1)	—

Continued on next page

Table 15.1 (continued)

Layer	Feature	Source Measure	Derivation	Expansion
1	signed_return_d	close (TSDB)	$\log(\text{close}_t/\text{close}_{t-1})$, shift(1)	—
NR	log_rk_d/w	rk (tick prices)	$\log$ [+ rolling mean 5d], shift(1)	—
NR	noise_gap_d/w	noise_gap	[rolling mean 5d], shift(1)	—
2	atm_iv_1m, _3m	atm_iv (Marquee)	shift(1)	lev/ Δ /z
2	vrp	atm_iv, rv	$IV^2 - RV$, shift(1)	lev/ Δ /z
2	skew_1m	skew (Marquee)	shift(1)	lev/ Δ /z
2	term_slope	atm_iv (3m, 1m)	$ATM_{3m} - ATM_{1m}$, shift(1)	lev/ Δ /z
2	butterfly_1m	skew, atm_iv	$0.5(IV_{25\delta P} + IV_{25\delta C}) - IV_{ATM}$, shift(1)	lev/ Δ /z
2	vvix	VVIX (TSDB)	shift(1)	lev/ Δ /z
2	iv_rv_gap	atm_iv, rv	$IV - \sqrt{RV \times 252}$, shift(1)	lev/ Δ /z
2	stock_atm_iv	EDRVOL (Marquee)	shift(1)	lev/ Δ /z
2	stock_vrp	stock_atm_iv, rv	stock $IV^2 - RV$, shift(1)	lev/ Δ /z
3	price_acceleration	E-mini mid-price	2nd derivative (win=50), daily agg, shift(1)	lev/ Δ /z
3	obi	E-mini L2 bid/ask	$(\Sigma \text{bid} - \Sigma \text{ask}) / (\Sigma \text{bid} + \Sigma \text{ask})$, daily agg, shift(1)	lev/ Δ /z
3	depth_ratio	E-mini L2 depth	$\log(\text{bid depth}/\text{ask depth})$, daily agg, shift(1)	lev/ Δ /z
3	spread_mean/std	E-mini bid/ask	mean/std spread (bps), shift(1)	lev/ Δ /z
3	vpin	E-mini trades	VPIN algorithm, shift(1)	lev/ Δ /z
3	kyle_lambda	E-mini trades	regress(Δmid , signed vol), shift(1)	lev/ Δ /z
4	treasury_slope	10y, 2y note prices	10y - 2y price proxy, shift(1)	lev/ Δ /z
4	fx_vol	USD/JPY, EUR/USD	annualized rolling RV (22d), shift(1)	lev/ Δ /z
4	commodity_vol	CL, GC (TSDB)	annualized rolling RV (22d), shift(1)	lev/ Δ /z
4	vix_level	VIX close (TSDB)	shift(1)	lev/ Δ /z
4	vix_futures_slope	VX1, VX2 (TSDB)	$VX2 - VX1$, shift(1)	lev/ Δ /z
4	dy_spillover	panel of RVs (34)	DY FEVD ($h=10, p=4$), shift(1)	lev/ Δ /z
5	fomc_proximity	FOMC calendar	days to next FOMC, shift(1)	—
5	nfp_proximity	NFP calendar	days to next NFP, shift(1)	—
5	opex_proximity	calendar math	days to next monthly OpEx, shift(1)	—
5	earnings_proximity	earnings calendar	days to next earnings, shift(1)	—
5	day_of_week	date	categorical encoding	—
5	month	date	categorical encoding	—
6	frac_diff_rv	rv	$(1 - L)^d$, $d \approx 0.35-0.45$, shift(1)	lev/ Δ /z
6	hurst_exponent	rv	rolling Hurst (22d), shift(1)	lev/ Δ /z
6	vol_of_vol	rv	std(RV) over 22d, shift(1)	lev/ Δ /z
6	regime_duration	rv	days since last 2σ spike, shift(1)	—
7	finbert_sentiment	news text	daily FinBERT score, shift(1)	lev/ Δ /z
7	negative_news_count	news text	count of negative articles, shift(1)	—

Notes. “NR” = noise-robust estimators computed from tick-level log prices, not 5-min bars. “Expansion” shows which features receive the {level, change, z -score} triple expansion for LightGBM (Chapter 8). Features marked “—” are used as-is. Rolling means compute the average in variance space first, then take log (?). **noise_gap** is a ratio, not log-transformed. Treasury slope uses note prices (US10YT=RR, US2YT=RR) as a proxy; true yield tickers are not available in TSDB. EUR/USD is sourced from Marquee FXIVOL, not TSDB. CL and GC require manual front-month contract roll (e.g. CLM26, GCM26); generic continuous contracts are not in TSDB. Deferred features (event-implied vol, sector-mean RV, cross-asset RV rank, WAP log returns, signed volume flow) can be added in later iterations. For layer-level summaries and feature counts per model family, see Chapter 9 Table 9.1.

Every Row Traces Back to Source

The feature matrix is the blueprint. To understand any feature, read across: the source measure tells you where the data comes from (Chapter 2), the derivation tells you the transformation chain, and the expansion column tells you what LightGBM sees.

Look-Ahead Lives in the Derivation Column

Every derivation must include `shift(1)` or equivalent. Any feature whose derivation does not include an explicit lag uses information from the forecast target period. The most subtle violations come from rolling windows that include day t when predicting day $t+1$.

Chapter 16

System Architecture

Chapter 11 presents prediction blending: two independent model branches combined at the forecast level. This chapter presents two alternative ensemble architectures, feature stacking and residual stacking, and compares all three.

16.1 Three Architectures

Figure 16.1 shows the data and prediction flow for each architecture side by side.

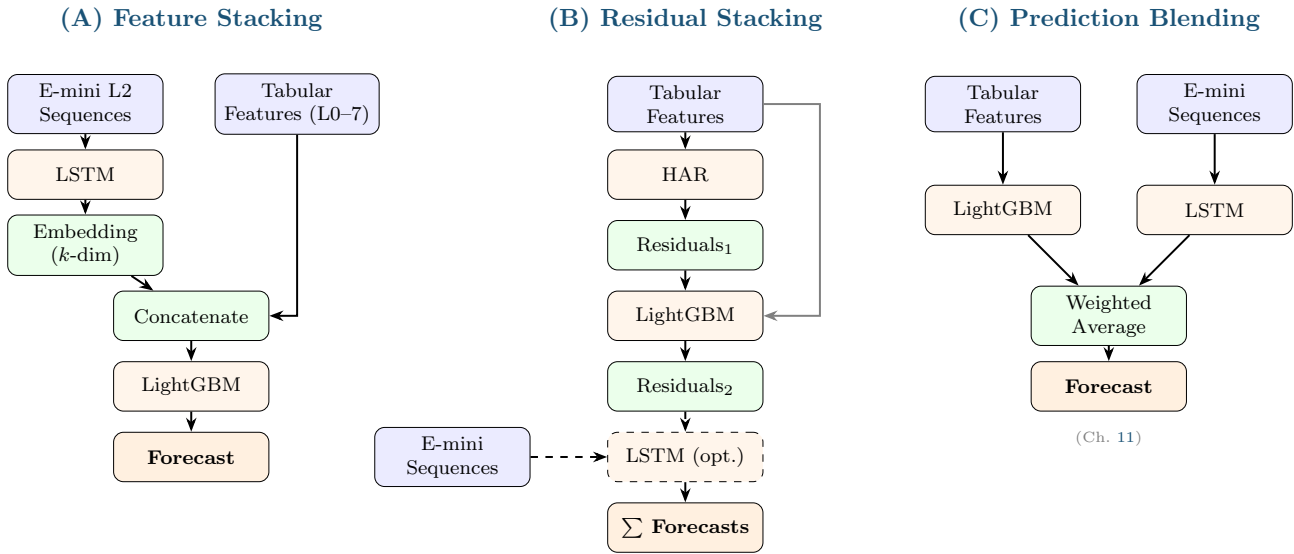


Figure 16.1: Three ensemble architectures. (A) LSTM embedding concatenated with tabular features for a single LightGBM. (B) Each model trains on residuals from the prior stage; final forecast is the sum. (C) Independent models blended at prediction level (Chapter 11).

16.2 Feature Stacking

The LSTM processes E-mini L2 5-min bars and LOB features (78 time steps per day) and produces a k -dimensional embedding vector (default $k = 32$, or $k = 1$ for a scalar forecast). This embedding is concatenated with the ~ 80 – 120 tabular features from Layers 0–7 to form a single expanded feature set. LightGBM then trains on the combined input.

The approach has one fundamental problem: LightGBM cannot back-propagate gradients through the LSTM, so the embedding is never optimized for the tree’s QLIKE objective. The LSTM learns representations that minimize its own loss, which may not be what the tree needs.

Table 16.1: Feature stacking: pros and cons.

Pros	Single training pass; LSTM learns representations the tree can exploit
Cons	Gradient isolation (tree cannot backprop into LSTM); embedding not optimized for tree objective; debugging harder; no RV literature demonstrates this beating alternatives

16.3 Residual Stacking

Stage 1: HAR baseline (OLS) produces a forecast and residuals. Stage 2: LightGBM trains on Stage 1 residuals with the full tabular feature set. Stage 3 (optional): LSTM trains on Stage 2 residuals from E-mini sequences. The final forecast is the sum of all stage forecasts.

Each model specializes by construction. HAR captures the multi-scale autoregressive structure that dominates at all horizons. LightGBM captures nonlinear patterns the HAR misses (regime interactions, jump-asymmetry effects). The LSTM, if used, captures whatever sequential dynamics remain in the residuals.

Table 16.2: Residual stacking: pros and cons.

Pros	Each model has a distinct role; no gradient isolation; clean residual targets; aligns with HARQ-X direction; supported by recent RV literature
Cons	Sequential training (each stage depends on prior); residual signal may be weak at later stages

16.4 Comparison

Table 16.3: Three-way architecture comparison.

Dimension	Feature Stacking	Residual Stacking	Pred. (Ch. 11)	Blending
Complexity	High (joint training)	Moderate (sequential)	Low (independent)	
Gradient flow	Broken (tree cannot backprop into LSTM)	Clean (residual targets)	N/A (independent)	
Literature	Weak (no RV paper)	Strong (HARQ-X, recent lit.)	Strong (??)	
Fallback	Must retrain tree without embedding	Drop Stage 3; keep HAR + LightGBM	Drop one model; keep the other	
Interpretability	Opaque (embedding)	Clear (stage contributions)	Clear (individual forecasts)	

Residual Stacking Gives Each Model a Distinct Role

HAR captures multi-scale RV persistence. LightGBM captures nonlinear patterns the HAR misses. LSTM (if used) captures whatever regime dynamics remain. Each model trains on residuals from the prior stage, so roles are distinct by construction.

No Evidence for Feature Stacking in RV Forecasting

No paper in the RV literature demonstrates LSTM-embedding-to-GBDT feature stacking beating prediction blending or residual stacking at any forecast horizon. The gradient isolation problem (Chapter 11) compounds this: embeddings are never optimized for the tree objective.

Chapter 17

Modular Pipeline Design

Chapter 14 gives the logical pipeline order. Chapter 16 describes the ensemble model architecture. This chapter describes the software design: config-driven, registry-based, every experiment defined by one YAML file.

17.1 Pipeline Architecture

Figure 17.1 shows the four pipeline stages and their plug points. TRAIN is the only modular stage; features, models, and CV strategies resolve from registries at runtime.

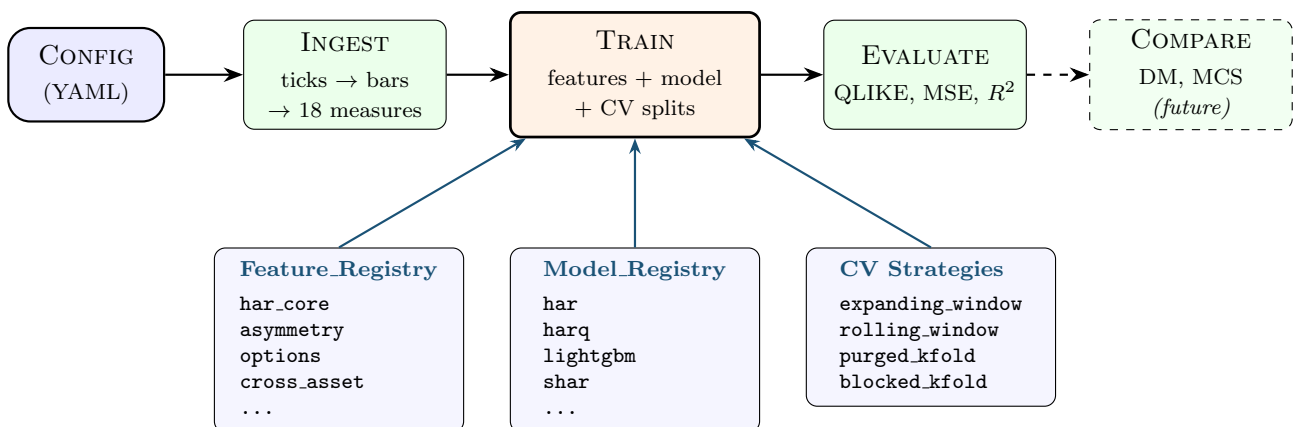


Figure 17.1: Pipeline architecture with plug points. CONFIG parameterises all stages. TRAIN resolves features, models, and CV strategies from registries at runtime. COMPARE (dashed) is not yet implemented.

17.2 Plug Points

17.2.1 Feature Layers

The `feature_layers` list in the config names the layers to compose. The pipeline iterates the list, resolves each name from `FEATURE_REGISTRY`, and calls `.compute()`. See Table 15.1 for the full feature inventory.

17.2.2 Models

The `model.name` field selects the model class from `MODEL_REGISTRY`. All models satisfy the same `VolModel` protocol: `.fit(X, y)` and `.predict(X)`. Model-specific hyperparameters go in `model.params`. See Chapter 9 for LightGBM configuration.

17.2.3 Cross-Validation

The `cv.method` field selects the splitter: expanding window, rolling window, purged k-fold, or blocked k-fold. Purge gap is set via `cv.purge_gap`. See Chapter 13 for evaluation methodology.

17.3 Configuration

Figure 17.2 shows two experiment configs side by side: a HAR baseline and a full LightGBM run.

HAR baseline	LightGBM with options features
<code>name: baseline_har</code>	<code>name: lgbm_full</code>
<code>universe: [SPY]</code>	<code>universe: [SPY, AAPL, MSFT, NVDA]</code>
<code>horizons: [1, 5, 22]</code>	<code>horizons: [1, 5, 22]</code>
<code>feature_layers: [har_core]</code>	<code>feature_layers:</code>
<code>model:</code>	<code>- har_core</code> <code>- asymmetry</code> <code>- options</code> <code>- cross_asset</code>
<code>cv:</code>	<code>model:</code>
<code> method: expanding_window</code>	<code> name: lightgbm</code>
<code> purge_gap: 5</code>	<code> params: {num_leaves: 31}</code>
	<code>cv:</code>
	<code> method: purged_kfold</code>
	<code> purge_gap: 22</code>

Figure 17.2: Two experiments, same pipeline. Left: HAR baseline with one feature layer. Right: LightGBM with four feature layers and stricter cross-validation.

One YAML, One Experiment

The entire experiment—universe, horizons, features, model, CV strategy—is defined by a single config file. No code changes needed to run a different experiment.

Registries Require Import

Feature layers and models register via decorators at import time. A new layer or model that is never imported will not appear in the registry. The package `__init__.py` triggers all registration imports.

Chapter 18

The Development Plan

Chapter 14 gives the logical order for layering features and models. This chapter gives the actual build order: what to implement first given project priorities and foundation work that must happen before anything else. Priority ordering: trading signal > academic rigor > model novelty.

18.1 Milestones

Table 18.1 defines seven milestones with acceptance criteria and dependencies.

Table 18.1: Development milestones. M1–M5 form the minimum viable deliverable.

M#	Milestone	Acceptance Criteria	Key Tasks	Deps
1	Fix Foundation	Purge gap $\geq h$ enforced; QLIKE sign matches ?; context kwarg added; all existing tests pass	CV fix, QLIKE fix, protocol extension, safe_log dedup	—
2	LightGBM	Custom QLIKE objective converges; Optuna finds improved params; walk-forward OOS predictions for 3 horizons	QLIKE gradient/hessian, model class, Optuna, walk-forward	M1
3	Tournament	8 models $\times$ 3 horizons table with DM p -values on dev universe	Run baselines, DM test, tournament table	M2
4	Layer 2 Options	Options features produce daily values with no look-ahead; QLIKE lift documented	OptionsLayer, IV surface wiring, validation	M1
5	Signal	IV–RV gap signal; equity curve; positive OOS Sharpe	Signal logic, P&L backtest, performance metrics	M3, M4
6	Ensemble	Residual stacking and prediction blending tested; 10-model tournament table	Residual stacking, inverse-QLIKE blending	M3
7	Stretch	Ordered by impact: regime QLIKE, MCS, LSTM feature, full universe, figures, Rashomon	Each task independent	M3–M6

18.2 Critical Path

Figure 18.1 shows milestone dependencies. The critical path (M1 $\rightarrow$ M2 $\rightarrow$ M3 $\rightarrow$ M6 $\rightarrow$ M7) is highlighted. M4 branches from M1 and runs in parallel with M2–M3. M5 and M6 can run in parallel after M3 completes.

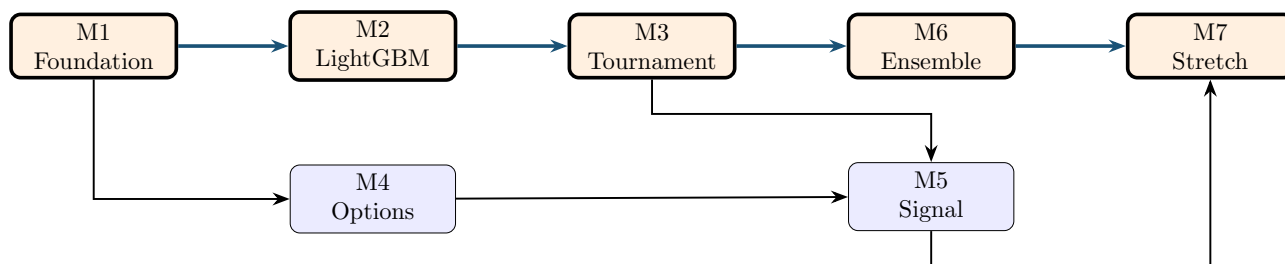


Figure 18.1: Milestone dependencies. Bold path: critical path (M1–M2–M3–M6–M7). M4 runs in parallel with M2–M3. M5 requires both M3 (forecasts) and M4 (options features). M7 waits on all prior milestones.

M1–M5 Is the Minimum Viable Deliverable

A QLIKE tournament (7 HAR variants + LightGBM, DM tests) plus a tradeable IV–RV signal with P&L backtest is a presentable result. Everything in M6–M7 is upside. If time runs out after M5, the project has a defensible outcome.

M1 Is Non-Negotiable

The purge gap bug causes silent data leakage for $h = 22$. The QLIKE sign convention determines whether the loss function penalizes over-prediction or under-prediction correctly. All results produced before M1 is complete are potentially invalid. Do not skip ahead.